

Introduction

Problem

The goal of this project was to experiment with sequential models and learn if they can predict short-term price direction of the DOGE/USDT cryptocurrency pair. Instead of forecasting the exact price, I made the task as a binary classification problem, and tried to solve it by building a hybrid **CNN-LSTM** architecture.

'Will the next candle close higher than the current one?' exploring if classification is simpler than regression, and direction is more relevant than magnitude.

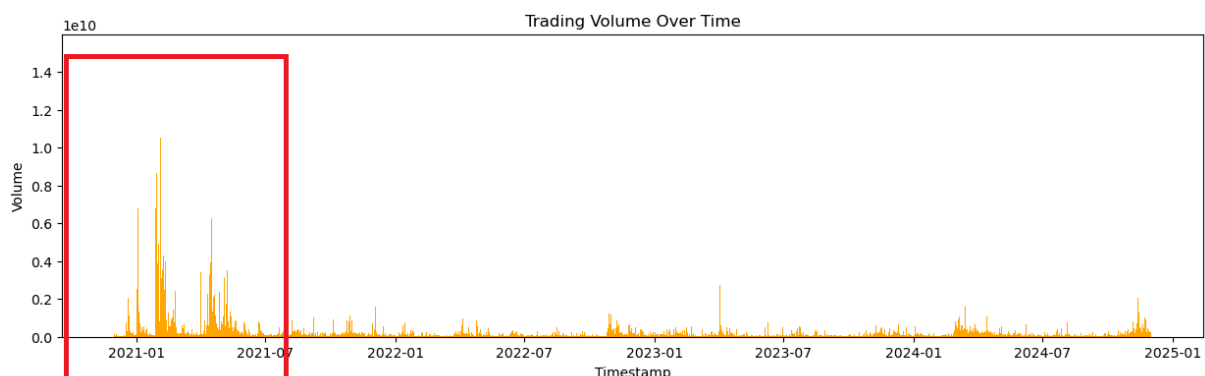
This project investigates how machine learning techniques can help detect short-term market behaviour. By building a binary prediction of bullish/bearish DOGE trends.

The motivation is both personal and technical. I have been interested in whether deep learning can provide useful insight into the market, and this work offers an opportunity to apply concepts learned during the course to a practical problem

Doge Coin was selected as the prototype due to its high volatility and active trading community. Its frequent price fluctuations make it an interesting benchmark for evaluating predictive models.

Dataset

I collected OHLCV data from 2020-2024, giving the last 4 years of 2 hours timesteps history covering multiple regimes, crashes and sideways markets. But this looked way too noisy to use, especially up until 07-2021. As we can see in the trading volume over time diagram below.



Which led me to drop this area from the dataset and choose the area between **07-2021** to **07-2024** in an attempt to exclude sudden drops and peaks. The sampling intervals of (*4h train, 1h eval*) were also dropped because 4h is deprecated from the API and multidimension architectures will take a longer time.

Klines API

Historical Candlestick data was collected from the Binance Klines API, with the following features per timestep: **Open** , **High**, **Low**, **Close**, **Volume**. Other 2 indicators were added later, called **RSI** for momentum and **ATR** for Volatility. The Target Label remained the same as from the pitch :

```
df["label"] = (df["next_close"] > df["close"]).astype(int)
```

And to capture a temporal structure, I used a window of 48 timesteps = 4 days avoiding the dilution of signal. The data presented low autocorrelations and the linear signal is nonexistent. The challenge here is entirely non-linear, which tells me that price direction may be difficult to infer solely from the OHLCV data, and that the hybrid strategy might be the right approach. More informations and details on the dataset in the([data/data.ipynb](#), [data_2y.ipynb](#))

Models Architecture

Several neural network architectures were implemented and evaluated to understand how different levels of model capacity affect predictive performance on short-term crypto price direction.

All the models share a common goal, which is to learn local price patterns and temporal dependencies from OHLCV data. In the diagram on the right are the simple visualisations for some of the classes that were experimented with. The code can be found in the project folder under the '[models/model_class.py](#)'.



The choices behind the models architecture

The fundamental structure that all the CNN-LSTM models share:

- **Feature Extractions:** using 1D convolution and max pooling to learn relevant spatial features(patterns) from the input time series sequence. The sequence of features(7/5 input, 48 steps) is processed.
- **LSTM block:** A type of Recurrent neural network to capture long-term temporal dependencies and sequential patterns in the data
- **Dense (output):** from the final timestep of the lstm is passed through one or more fully connected layers (nn.layers) and a final sigmoid activation to produce a probability output (0 for bearish/ 1 for bullish).

The model development was a three-phase process focused on building increasingly sophisticated hybrid architecture for time-series prediction.

- **Phase 1 ([optimised.ipynb](#)):** Basic and basic hybrid using 5 essential financial features(OHLCV).
- **Phase 2 ([2hours_optimised.ipynb](#)):** introducing the key enhancement by adding 2 technical indicators to create 7-features input, using three architectures
- **Phase 3 ([tunes.ipynb](#)):** Hyperparameter tuning of LR and final optimisation, 7 features, adding L2 regularization and tunable LSTM hidden layers.

The training loop, defined by `train_and_evaluate` function, also been adjusted to the models and tests but in each phase it was constant with Binary Cross-entropy Loss (**nn.BCELoss**) because it penalizes confident *wrong* predictions, **Adam** optimizer for efficient gradient descent, **50 epochs** in all phases, evaluated on the test set using **accuracy**, **F1 Score**, and **AUC**(area under the ROC curve).

In the /src folder in the code, the 3 phases are implemented with the use of the mentioned classes and other plotting tools to process and visualise the work.

7. Results

The result of the 3 phase training & evaluation process clearly illustrates the evolution of the model performance, guided by increasing the data richness and architectural capacity tuning.



Phase 3 shows divergent behaviours when manipulating hyperparameters. While some models remained stable, the aggressive tuning (LR=0.1) led to loss increase, meaning that the models' learning process collapsed when pushed too hard. This highlights the fragility of the learning process on this noisy dataset. The comparison of key performance metrics in the diagram below across the phases confirms that the predictive signal is weak as well.

Phase 1 Optimized Models Original LSTM	Phase 2 2-Hour Optimized 2hCNN-LSTM v03	Phase 3 Tuned Models Tuned (LSTM=128, L2=1e-4)
Accuracy	52.33%	52.10%
F1 Score	0.5012	0.5173
AUC	0.5249	0.5216
Best Loss	0.6928	0.6906
		52.33%
		0.5251
		0.5236
		0.6928

8. Discussion & Reflection

Faint Signal Problem: The biggest struggle I faced was the realisation that the predictive signal is almost nonexistent within the selected OHLCV, RSI, ATR for price detection. I thought I would notice at least a noticeable improvement. Despite the complexity of the model, the best result of AUC was still about 0.523. Which is basically barely better than flipping a coin, if seen from an outsider's view.

The decision to not test on new, unseen data means the final model's generalization ability remains unproven. This step was skipped because the test set results were too weak, indicating that the model has a high variance and would likely fail outside of a highly optimized environment. Besides, the market is controlled by Elon Musk's tweets.

What I learned: The project confirmed that the hybrid CNN-LSTM structure is appropriate for the sequential data, successfully utilizing the increased input from 7 features. However, it challenged the hope that technical indicators alone are sufficient for prediction. The improvement between phases demonstrated that the signal is faint, requiring higher architectural sensitivity just to detect it, but it is not robust enough for real-world application.

Future work: To achieve meaningful results, the focus must shift from tuning the existing model to expanding the scope of data and architecture. Maybe starting with an established coin (Bitcoin or Eth) and including exogenous data, such as NLP-derived sentiment or advanced features like attention mechanisms into the model, to prioritize the most informative timesteps and features.